

# Dawnwood v0.5: universality assessment

**Assessment revision 1 - 23 September 2026 Target:** unchanged numerical profile DWI-N1-0.5 / kernel 0.5.0.

**Requested scope:** investigate the existing kernel without changing it. Prepared for Tom Klootwijk, Dawnwood Interactive.

## Finding

**The literal, closed v0.5 implementation is not Turing-universal under an unbounded-computation definition.** Its complete stored configuration is finite, its record counts have fixed upper bounds, and its runtime refuses to advance beyond the uint32 epoch limit. This is an implementation theorem, not merely the absence of a successful benchmark.

**That does not disprove universality of a suitably defined, unbounded Dawnwood architecture.** Such a model and its encoding/simulation argument have not been supplied. The distinction also applies to ordinary physical computers: an actual finite machine is different from its idealized extensible-memory computational model.

There is a further structural result: disabling feedback makes the wavefront records evolve independently. Enabling feedback restores real communication through the operator LUT. New observations demonstrate both behaviors on the unchanged CPU and GTX executable, with exact matching checkpoints.

| Question                                                                               | Finding                                                                                                                     |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Can the closed v0.5 executable simulate arbitrary unbounded computations?              | No, under the configuration-faithful meaning stated below: finite stored state and an enforced epoch horizon prohibit it.   |
| Do more lanes alone make the feedback-off recurrence universal?                        | No. Lanes do not exchange information; its fixed-precision repeated numerical map has a uniform eventual-periodicity bound. |
| Is feedback-enabled Dawnwood incapable of universality under every idealization?       | Not established. The finite-executable theorem and feedback-off theorem do not prove that broader negative claim.           |
| Has a universal encoding into the existing operator/state recurrence been established? | No. Program/input encoding, an invariant representation, step correspondence and output/halting decoding remain absent.     |
| Does incomplete algorithm self-rewriting independently disprove universality?          | No. A universal machine can have a fixed transition rule.                                                                   |

The assessed v0.5 release, its source, shaders and binaries remain unchanged. This assessment and its evidence are separate artifacts; they do not rename the kernel or introduce a new simulation mode.

## 1. Meaning of the claim

Here a computational universality claim requires an effective encoding of machine and input, and a fixed interpretation of resulting states, such that the target's computation is faithfully reproduced. A simulation must support arbitrarily long finite prefixes and the distinction between termination and continued computation. Encoding the answer in advance, letting an external program execute the computation, or interpreting a time counter with a universal interpreter is not evidence that the kernel performs it.

Turing's universal-machine construction separates a finite control table from the tape and the encoded machine description. That supplies the reference computational notion, rather than a requirement for a particular instruction vocabulary. [Turing, 1936, sections 1, 5-7.](#)

The negative results below concern fixed, memoryless decoding of the represented configuration. Where an idealization discards a counter or metadata, the decoder must discard it too. No live host edits, streamed external memory, arbitrary-precision oracle, retained unbounded observation history or externally supplied computation is included in the closed-kernel model.

Computational universality is also distinct from *intrinsic universality*, which concerns structure-preserving simulation of other cellular automata through specified space/time encodings. This report does not claim the stronger property for Dawnwood. [Ollinger, 2009.](#)

## 2. Actual transition, not a replacement machine

Write the epoch-boundary snapshot as  $(c, w, o)$ : 64-byte configuration,  $N$  wavefront records and  $M$  operator records. Each wavefront is 128 bytes; each operator is 64 bytes. At epoch  $e$ , the code executes:

```
a_i = (17*i + e) mod N
b_i = (31*i + e + 1) mod N
o'_i = mutate(o_i, o_30, o_5, w_(a_i), w_(b_i), c)
w'_j = evolve(w_j, c, j, o')
e' = e + 1
```

All mutation invocations read the old population and old operators. Each evolution invocation reads its own old wavefront and the new LUT. Global exchange occurs after the complete epoch. Bounded dispatch partitions and the ARM stage split implement this ordering; they do not introduce extra persistent memory or independent tapes.

| Mechanism                   | What exists in v0.5                                                                                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Persistent wavefront memory | 28 FP32 values and four uint32 values per record, including channels, history, inverse, RNG, orientation and route.                                                    |
| Persistent operator memory  | 12 FP32 values and four uint32 words, stored in an actual integer texture.                                                                                             |
| Scalar body                 | Eight four-bit instructions: identity, add/subtract gain, multiply coupling, sine, cosine, absolute value, negate and square.                                          |
| Routing                     | A root-to-leaf binary traversal restarted each epoch, with configured depth at most 30. It is not an explicitly programmable instruction pointer.                      |
| Communication               | State-to-operator sampling in mutation, followed by operator-to-state influence in evolution. There are no direct reads of neighboring wavefront records in evolution. |
| Population and catalogue    | Allocated at initialization/resume; counts do not grow during advancement.                                                                                             |

These are implementation facts, not independent impossibility proofs. In particular, a short instruction set, fixed rules, a fixed local neighborhood or a lack of self-modification can coexist with computational universality in a properly scalable system. Cook's Rule-110 construction is a concrete example of a fixed rule with a nontrivial encoding proof; that result is not transferred to Dawnwood. [Cook, 2009.](#)

## 3. Theorem: finite configuration and execution horizon

**Assumptions.** Use a fixed deterministic backend transition and its specified finite-word operations. Fix the v0.5 ABI and allowed interfaces. Input is a validated finite snapshot. No external edits or supplementary memory are introduced during its closed evolution. The theorem concerns logical completed-epoch configurations, not GPU timing or the driver's temporary storage.

**Finite configuration bound.** For fixed  $N$  and  $M$ , every logical snapshot can be encoded in

$$B(N,M) = 512 + 1024*N + 512*M \text{ bits.}$$

$$\text{Number of possible snapshots} \leq 2^{B(N,M)}.$$

Proof: concatenate the configuration and record words. Validation, dependent fields and unreachable values reduce the represented set; they cannot increase it. Scratch and ping-pong storage do not furnish independent persistent input at the next logical epoch: they are implementation storage for this same transition.

The family of all ABI-permitted counts is also finite. Both counts are uint32, so with  $U = 2^{32} - 1$ , padding each array to  $U$  entries yields a loose injective representation with

$$B_{\max} = 512 + 1536*U = 6,597,069,765,632 \text{ bits.}$$

Counts in the configuration distinguish differently sized snapshots. This deliberately loose mathematical upper bound is not a claim that the hardware can allocate that payload. Device limits and available memory are much stricter.

**Epoch horizon.** Starting from stored epoch  $e$ , the unchanged public runtime permits at most  $U-e$  further successfully completed epochs. CPU stepping rejects  $e=U$ ; Vulkan advancement and CLI execution reject overflowing requests. Checkpoint files retain  $e$ , so ordinary resume does not remove the limit. The last permitted epoch is  $U$ , not  $U-1$ .

**Consequence.** The unchanged closed implementation cannot represent and simulate all configurations of arbitrary unbounded Turing computations. Its global input/configuration domain is finite, and it has no unbounded execution horizon through its defined interface. Under a fixed configuration decoder, repeated finite stored states could not encode an unbounded sequence of distinct target configurations even if repetition were allowed.

This does not show that a finite machine is computationally useless or incapable of particular bounded simulations. Nor does it decide a different model in which addresses, record precision, population or available tape can grow without a fixed bound. Removing those restrictions would require stating that different model explicitly; the present theorem must not be silently reused for it.

## 4. Theorem: feedback-off noninterference

With  $C.\text{feedback}=0$ ,  $\text{dw\_mutate}$  immediately returns the old operator. Consequently  $O$  remains constant. Numerical evolution receives  $w_j$ , the fixed LUT/configuration and lane  $j$ . It reads no  $w_i$  for  $i$  different from  $j$ . The lane number is only assigned to  $\text{alpha\_ref}$ , which numerical evolution does not subsequently consume; epoch and population count do not enter the feedback-off numerical update.

Thus, for fixed LUT/configuration and a given lane,

$$w_j(t+1) = F_j(w_j(t)).$$

**Noninterference statement.** Consider two populations with equal configuration and LUT, equal initial state in lane  $j$ , and any differences in other lanes. At every epoch for which both computations successfully advance, their lane- $j$  states are equal.

Proof by induction: equality holds at initialization. The two next-lane evaluations receive identical arguments, so produce identical results under the fixed backend transition. Repeat. An invalid or failed population may stop its whole runtime, which is why the statement concerns common successfully defined epochs.

This comparison holds population size and initial lane values fixed. Changing CLI `--count` regenerates different initial coordinates and is not a valid noninterference experiment.

**Conditional periodicity corollary.** Consider only the mathematical iteration of these fixed feedback-off numerical maps, hypothetically repeated beyond the wrapper's cap, at unchanged finite record precision. Exclude any new unbounded counter from the configuration read by the decoder. On every indefinitely defined trajectory, a lane

enters a cycle after at most  $K-1$  transient steps, with cycle length at most  $K$ , for the generous bound  $\kappa = 2^{1024}$ . This follows from the pigeonhole principle on its finite record alphabet.

Let  $L = \text{lcm}(1, 2, \dots, K)$ . Every lane then repeats after  $L$  steps once the common transient bound is reached. The product population repeats after  $L$  too, even if arbitrarily many such independent lanes are admitted, because  $K$  bounds every lane. This argument does not require all lanes to follow the same fixed map.

A fixed, memoryless decoder of this projected population must also repeat. It therefore cannot faithfully reproduce a Turing-machine orbit with infinitely many distinct configurations. The numerical projection's periodicity says nothing about a newly added, observable unbounded clock or a stateful external decoder. If a trajectory becomes invalid, indefinite computation fails instead; no closure of all extreme FP32 inputs is presumed.

**Important switch distinction:** `mutation=0` does not freeze the LUT. Seed updates, jitter-dependent opcode toggles and other drift terms can remain active. The proved factorization uses `feedback=0`.

## 5. What feedback and self-modification do establish

With feedback enabled, the factorization in section 4 does not hold. A wavefront can change a sampled operator, and that changed operator can affect other wavefronts. The dependency indices in section 2 identify the actual communication path. Sparse sampling alone is not a proof of nonuniversality; information can propagate over multiple epochs.

Automatic scalar-program mutation is precisely restricted. The upper seven opcode nibbles remain constant. In the lowest nibble, only `1 <-> 2` and `4 <-> 5` toggles occur; all other initially valid opcode classes remain fixed. Therefore each operator's program word has at most two reachable values under automatic mutation, and  $M$  operators have at most  $2^M$  such program-word arrangements. This bound excludes external authoring edits and says nothing comparable about the varying FP32 parameters or wavefront state.

The bound does **not** refute universality: a universal machine can keep its entire instruction table fixed. Likewise, missing native-algorithm rewriting is an architectural limitation separate from universality. The relevant unanswered question is whether these actual recurrent transitions preserve an encoding capable of arbitrary computation in a precisely specified scalable model.

No invariant Boolean/tape encoding, step-simulation relation or universal program compiler was found in the existing source, documentation or retained evidence. That is an evidence finding, not a proof that none can exist for every possible idealization. Isolating a useful arithmetic identity inside `dw_body` would not be enough: the representation must survive geometry, mutation, the complete evolution and the next feedback interval.

## 6. New observations on the unchanged executable

These are manual invocations of the existing `run` command, with isolated input-checkpoint edits. No kernel, shader, build or test source was changed. No phone rerun or new performance claim is made in this assessment.

The starting population contains 64 states and 31 operators. Each pair starts from byte-identical snapshots except the specified state-0 field; feedback is set in the stored configuration before resume. Both CPU and actual GTX paths execute 16 epochs. Counts below describe differing final records between the original and perturbed inputs, not CPU/GPU mismatches.

| Initial perturbation                      | Feedback | Changed wavefront records | Changed operator records |
|-------------------------------------------|----------|---------------------------|--------------------------|
| <code>state[0].history: 0 to 1</code>     | Off      | 1: state 0                | 0                        |
| <code>state[0].history: 0 to 1</code>     | On       | 1: state 0                | 9                        |
| <code>state[0].rng: toggle low bit</code> | Off      | 1: state 0                | 0                        |
| <code>state[0].rng: toggle low bit</code> | On       | 64: all states            | 31: all operators        |

CPU and GTX produce the same rows. Seven paired final checkpoints are byte-identical across those backends: six 16-epoch cases (baseline/history/RNG for each feedback setting), plus the epoch-boundary case below. All successful runs report healthy records. GTX core/synchronization validation is enabled and records zero errors and warnings.

The history perturbation's limited spread is retained alongside the stronger RNG result. Feedback can transmit information, but this does not promise that every perturbation will reach every record or implement a useful logical gate. These observations support the implementation interpretation; source reasoning supplies the general noninterference proof.

**Epoch boundary observation.** A zero-step checkpoint was edited to store epoch 4,294,967,294; this is synthetic positioning near the boundary, not billions of executed updates. One actual CPU epoch and one actual GTX epoch each complete at 4,294,967,295, yielding identical checkpoints. A further resumed CLI run on each selected backend exits with code 1 and Requested epochs exceed the uint32 epoch ABI. The rejection happens in CLI preflight. Separate CPU/Vulkan internal guards are established by source inspection, not by falsely attributing this preflight rejection to a launched shader.

Raw commands, input edits, exit codes, stdout/stderr, checkpoint hashes and changed-record lists are in `evidence/`. `evidence/observations.json` consolidates the results without replacing those records.

## 7. Evidence still needed for a broader claim

An affirmative claim about a scalable architecture requires all of the following, rather than additional ordinary benchmarks:

1. A precise model that states what may grow, how addresses and time are represented, which arithmetic semantics apply, and how it relates to the fixed v0.5 executable.
2. An effective, uniform encoding of arbitrary machine/program and input. The encoding must not precompute the requested answer or depend on an unknown halting time.
3. A preserved representation of control and writable memory through the complete Dawnwood transition, including active feedback when used. FP32 thresholds and perturbations need exact invariants or stated error bounds; sampled numerical agreement is insufficient.
4. A step-correspondence argument: decoded states follow the target transition with a specified finite simulation overhead, and arbitrarily long finite target prefixes remain representable.
5. Fixed output and halting decoding, plus a stated resource-scaling argument. A host loop that interprets or rewrites the computation must be counted as part of a different combined machine.

For literal unchanged v0.5, the finite-domain and epoch restrictions already prevent this unbounded claim. For an unspecified scalable feedback-enabled model, the right status remains **unproved**, rather than either “universal” or “impossible.” This assessment supplies a negative result for the exact implementation and a stronger feedback-off restriction; it does not manufacture a positive theorem by adding a separate universal automaton.

## 8. Audited sources and identity

Paths below are relative to the existing `Dawnwood_GPU_v0.3` working directory, whose historical folder name contains the released v0.5 files.

| Source location                                                                       | Relevant fact                                                                                   |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>include/numeric_types.inc:44-64;</code><br><code>include/numeric.hpp:5-8</code> | Finite record members and asserted 128/64/64-byte persistent ABI.                               |
| <code>src/cpu.cpp:5,9-11</code>                                                       | Count/depth validation, epoch check, old-LUT mutation and per-lane evolution.                   |
| <code>include/numeric_types.inc:122-158</code>                                        | Scalar-body interpreter, feedback early return, parameter updates and program-nibble invariant. |
| <code>include/numeric_evolve.inc:42-141</code>                                        | Actual routing and full evolution; own-state inputs and <code>alpha_ref</code> assignment.      |
| <code>src/driver.cpp:34-35,87</code>                                                  | Checkpoint epoch storage/restoration and CLI overflow preflight.                                |
| <code>src/vulkan.cpp:134-169</code>                                                   | GPU mutation/evolution ordering, chunking and epoch limit.                                      |
| <code>docs/FORMALIZATION_v0.5.md;</code><br><code>docs/CLAIMS.md</code>               | Existing numerical bindings and separation of universality from algorithm self-rewriting.       |

The investigated executable SHA-256 is:

`f208a48e4cd6c4cad9403736cbbba398c8d283649c1a025418521c3d9de1bb10`

The released shader-source fingerprint is:

`3b89050cd591cdb79f670f19a201b6e3feebb1cf0c44ffc138528d63958314fd`

The v0.5 manifest is rechecked after the investigation to establish that the assessed release remains unchanged. `evidence/release_integrity.json` records that result and the separate assessment's provenance. The source PDFs and published v0.5 PDF are preserved; this is assessment revision 1, not a new kernel version.